Title: Write a jsp servlet program to demonstrate session handling using url rewriting hidden, formfeild, cookies session. provide me write up for this

Below is a **lab-manual style write-up** for the experiment, covering all four session-handling techniques: **URL Rewriting, Hidden Form Fields, Cookies, and HttpSession**.

Experiment: Session Handling in JSP/Servlet

Title

Write a JSP/Servlet Program to Demonstrate Session Handling Using URL Rewriting, Hidden Form Fields, Cookies, and HttpSession

Aim

To develop a JSP/Servlet application that demonstrates different techniques for maintaining user session information, namely **URL Rewriting, Hidden Form Fields, Cookies, and HttpSession**.

Objectives

1. To understand the concept and importance of session tracking in web applications.
2. To implement session handling using URL rewriting, hidden form fields, cookies, and HttpSession.
3. To understand how user information can be maintained across multiple HTTP requests.

Software/Tools Required

Software/Tool	Purpose
JDK	Java development environment
Apache Tomcat	Servlet/JSP container
Eclipse / IntelliJ IDEA / NetBeans	Development environment
HTML/JSP	User interface
Java Servlet	Server-side processing
Web Browser	To execute and test the

Software/Tool	Purpose
	application

Theory

HTTP is a **stateless protocol**, which means that the web server does not automatically remember information about a user between two successive requests. **Session tracking** techniques are therefore used to maintain user-specific information throughout a web session.

The commonly used session tracking mechanisms in JSP/Servlet are:

1. URL Rewriting

In URL rewriting, session-related information is appended to the URL. It is useful when cookies are disabled in the user's browser.

Example:

```
welcome.jsp?username=Pratibha
```

In Java Servlet applications, response.encodeURL() can be used to encode URLs for session tracking.

2. Hidden Form Fields

A hidden field is included inside an HTML form. The value is not displayed to the user but is submitted to the server along with the form data.

Example:

HTML

```
<inputtype="hidden"name="username"value="Pratibha">
```

3. Cookies

A cookie is a small piece of information stored in the user's browser. The server can create a cookie and retrieve it in subsequent requests.

Example:

Java

```
Cookiecookie=newCookie("username", "Pratibha");
response.addCookie(cookie);
```

4. HttpSession

HttpSession is the most commonly used session management mechanism in Servlet applications. The server creates a session object for a user and stores information in it.

Example:

Java

```
HttpSessionsession=request.getSession();
session.setAttribute("username", "Pratibha");
```

Procedure

1. Install JDK and Apache Tomcat.
2. Create a Dynamic Web Project in the preferred IDE.
3. Create a JSP page to accept the username.
4. Create separate Servlets to demonstrate:
    - URL Rewriting
    - Hidden Form Fields
    - Cookies
    - HttpSession
5. Pass the username from one request to another using each session-tracking mechanism.
6. Deploy the application on Apache Tomcat.
7. Run the application through a web browser.
8. Verify that the username/session information is maintained across different requests.
9. Compare the working of the four session-tracking techniques.

Sample Program Structure

```
SessionHandling/
│
├── WebContent/
│   ├── index.jsp
│   ├── urlRewrite.jsp
│   └── hiddenForm.jsp
│
└── Java Resources/
    └── src/
        ├── URLRewriteServlet.java
        ├── HiddenFieldServlet.java
        ├── CookieServlet.java
        └── SessionServlet.java
```

1. JSP Page – index.jsp

JSP

```
<!DOCTYPE html><html><head><title>Session
Handling</title></head><body><h2>Session Handling using
JSP/Servlet</h2><formaction="session"method="post">
   Enter Username:
   <inputtype="text"name="username"required><inputtype="submit"value="Submit"></
form></body></html>
```

2. URL Rewriting

Java

```
importjava.io.*;
importjakarta.servlet.*;
importjakarta.servlet.http.*;

publicclassURLRewriteServletextendsHttpServlet {

   protectedvoiddoGet(HttpServletRequestrequest,
                      HttpServletResponseresponse)
                      throwsServletException, IOException {

      response.setContentType("text/html");
      PrintWriterout=response.getWriter();
```

```
    Stringusername=request.getParameter("username");

    out.println("<h2>URL Rewriting</h2>");
    out.println("Welcome, "+username+"<br>");

    Stringurl=response.encodeURL(
       "URLRewriteServlet?username="+username);

    out.println("<a href='"+url+"'>Continue</a>");
  }
}
```

3. Hidden Form Field

Java

```
importjava.io.*;
importjakarta.servlet.*;
importjakarta.servlet.http.*;

publicclassHiddenFieldServletextendsHttpServlet {

  protectedvoiddoGet(HttpServletRequestrequest,
                     HttpServletResponseresponse)
                     throwsServletException, IOException {

    response.setContentType("text/html");
    PrintWriterout=response.getWriter();

    Stringusername=request.getParameter("username");

    out.println("<h2>Hidden Form Field</h2>");

    out.println("<form action='HiddenFieldServlet' method='post'>");
    out.println("<input type='hidden' name='username' value='"+username+"'>");
    out.println("<input type='submit' value='Continue'>");
    out.println("</form>");
  }

  protectedvoiddoPost(HttpServletRequestrequest,
                      HttpServletResponseresponse)
                      throwsServletException, IOException {
```

```
    response.setContentType("text/html");
    PrintWriterout=response.getWriter();

    Stringusername=request.getParameter("username");

    out.println("<h2>Welcome, "+username+"</h2>");
  }
}
```

4. Cookies

Java

```
importjava.io.*;
importjakarta.servlet.*;
importjakarta.servlet.http.*;

publicclassCookieServletextendsHttpServlet {

  protectedvoiddoGet(HttpServletRequestrequest,
                HttpServletResponseresponse)
                throwsServletException, IOException {

    response.setContentType("text/html");
    PrintWriterout=response.getWriter();

    Stringusername=request.getParameter("username");

    Cookiecookie=newCookie("username", username);
    response.addCookie(cookie);

    out.println("<h2>Cookie Session Handling</h2>");
    out.println("Cookie created successfully.<br>");
    out.println("<a href='ReadCookieServlet'>Read Cookie</a>");
  }
}
```

5. HttpSession

Java

```
importjava.io.*;
importjakarta.servlet.*;
importjakarta.servlet.http.*;

publicclassSessionServletextendsHttpServlet {

   protectedvoiddoPost(HttpServletRequestrequest,
                       HttpServletResponseresponse)
                       throwsServletException, IOException {

      response.setContentType("text/html");
      PrintWriterout=response.getWriter();

      Stringusername=request.getParameter("username");

      HttpSessionsession=request.getSession();

      session.setAttribute("username", username);

      out.println("<h2>HttpSession</h2>");
      out.println("Session created successfully.<br>");
      out.println("Welcome, "+session.getAttribute("username"));
   }
}
```

Expected Output

When the application is executed:

```
Session Handling using JSP/Servlet

Enter Username: Pratibha

[Submit]
```

The application demonstrates:

```
1. URL Rewriting
  Username is passed through the URL.

2. Hidden Form Field
  Username is transferred through a hidden input field.
```

3. Cookies
   Username is stored in the browser as a cookie.

4. HttpSession
   Username is stored on the server using HttpSession.

Comparison

Technique	Storage	Visible to User	Works Without Cookies
URL Rewriting	URL	Yes	Yes
Hidden Form Field	HTML Form	No	Yes
Cookies	Browser	No	No
HttpSession	Server	No	Yes*

HttpSession commonly uses a session ID stored in a cookie; if cookies are disabled, URL rewriting can be used to carry the session ID.

Result

The JSP/Servlet application was successfully developed to demonstrate **session handling using URL rewriting, hidden form fields, cookies, and HttpSession**. The experiment demonstrates how session information can be maintained across multiple HTTP requests in a web application.

Learning Outcomes

After completing this experiment, students will be able to:

1. **Understand** the need for session tracking in stateless HTTP communication.
2. **Implement** different session management techniques using JSP and Servlets.
3. **Compare** URL rewriting, hidden form fields, cookies, and HttpSession based on their working and limitations.